University of Niš
Faculty of Sciences and Mathematics
Department of Computer Science
Doctoral Academic Studies in Computer Science

Seminar paper
Course: Machine Learning (20.IDI12)

The EM Algorithm for Gaussian Mixtures

Mentor
Prof. Branimir Todorović

Student
Elvir Muslić
ORCID 0009-0006-4042-6550

Niš, October 2026

Abstract

A Gaussian mixture models data that fall into groups without labels: each group is a Gaussian, and the group of each point is hidden. Its log-likelihood has no closed-form maximizer, and the expectation-maximization (EM) algorithm climbs it in two closed-form steps. The E-step computes, by Bayes' rule, the responsibility of each component for each point, and the M-step refits each component with these responsibilities as weights. This paper presents the model and the algorithm, works through one iteration by hand, and explains why no iteration lowers the log-likelihood. The estimator `GaussianMixture` of scikit-learn reproduces the hand iteration to $6.2 \cdot 10^{-15}$. On the Old Faithful data, EM from a poor start first makes the two components almost identical, leaves this state slowly, and converges after 53 iterations; twenty random starts all reach the same log-likelihood of the standardized data, -385.4607.

Contents

1. Introduction 2
2. The Gaussian Mixture 3
3. The EM Algorithm 4
4. Why EM Works 6
5. Experiments 7
6. Conclusion 9

Code availability 9

References 10

1. Introduction

The Old Faithful geyser in Yellowstone National Park erupts at irregular intervals. The data set `faithful` of R records 272 eruptions, each by its duration and by the waiting time to the next eruption, in minutes; Azzalini and Bowman [1] analyze a more complete version of this record. Plotted, the eruptions fall into two groups, short eruptions followed by short waits and long ones followed by long waits (Figure 1(d)). No eruption is labeled with its group, and finding the groups from the points alone is the task of clustering.

A model makes the task precise. A Gaussian mixture describes each group by a Gaussian distribution, with its own center, spread and share of the data, and gives every point a probability of belonging to each group instead of a hard label. Its parameters are fitted by maximum likelihood, which has no closed-form solution here, no formula that gives the parameters directly, because the labels are missing. The expectation-maximization (EM) algorithm of Dempster, Laird and Rubin [2] fills the labels in with probabilities, refits the groups, and repeats [3, 4]. The paper does four things.

(1) Section 2 presents the Gaussian mixture and its log-likelihood.
(2) Section 3 presents the two steps of EM and its algorithm, and carries out one iteration by hand.

(3) Section 4 explains why no iteration lowers the log-likelihood, and where EM can stop.
(4) Section 5 runs EM with scikit-learn [5] on Old Faithful, checked against the hand iteration.

2. The Gaussian Mixture

2.1. **The Gaussian.** A Gaussian, or normal, distribution in $\mathbb{R}^d$ has two parameters. Its *mean* $\mu \in \mathbb{R}^d$ is the center of its points. Its *covariance* Σ, a symmetric $d \times d$ matrix, with a positive variance in every direction, describes their spread: the diagonal holds the variance of each coordinate, and an off-diagonal entry says how two coordinates vary together, positive when they tend to grow together.

The density $\mathcal{N}(x \mid \mu, \Sigma)$ is proportional to $\exp\bigl(-\frac{1}{2}(x-\mu)^{\mathsf{T}}\Sigma^{-1}(x-\mu)\bigr)$: it is largest at μ and falls off with the distance from μ, measured in units of the spread in each direction. Points of equal density lie on ellipses centered at μ (ellipsoids if $d > 2$), stretched along the directions in which the points spread most; Figure 1 draws such an ellipse for each component.

2.2. **The mixture and its hidden labels.** A Gaussian mixture with K components has the density

$$p(x \mid \theta) = \sum_{k=1}^{K} \pi_k \, \mathcal{N}(x \mid \mu_k, \Sigma_k), \tag{2.1}$$

where the weights satisfy $\pi_k \geq 0$ and $\pi_1 + \cdots + \pi_K = 1$, and θ collects all weights, means and covariances. In words, the density of a point is a weighted average of K Gaussian densities, one per group, and the weight π_k is the share of the data that component k describes.

The mixture is the density of a two-stage experiment. First a label $z \in \{1, \ldots, K\}$ is drawn, with probability $P(z = k) = \pi_k$; then x is drawn from the Gaussian of component z. Summing over the label gives (2.1). The data record x and lose z: the label is *hidden*, or latent. For Old Faithful, $K = 2$ and $d = 2$, and the hidden label of an eruption says whether it is a short or a long one.

2.3. **The log-likelihood.** For n independent points $x_1, \ldots, x_n$, the log-likelihood of the parameters is

$$\ell(\theta) = \sum_{i=1}^{n} \log \sum_{k=1}^{K} \pi_k \, \mathcal{N}(x_i \mid \mu_k, \Sigma_k). \tag{2.2}$$

In words, $\ell(\theta)$ measures how probable the data are under the mixture with parameters θ; the maximum-likelihood estimate is the θ that maximizes it. For a mixture, ℓ has no maximum: if one component shrinks onto a single data point, its density there grows without bound, and so does ℓ [3, Section 9.2]. The estimate sought is the best local maximum with no collapsed component.

If the labels were known, the inner sum would keep a single term for each point, that of its label, and the logarithm would split ℓ into separate parts, one per component. The maximizer would then be the ordinary fit of each group: its share of the points, its sample mean and its sample covariance.

In (2.2), however, the logarithm is applied to a sum. Setting the derivative with respect to μ_k to zero gives μ_k as an average of the points, weighted by how much each point belongs to component k; but these weights depend on all the parameters, μ_k included, and so do the equations for π_k and Σ_k [3, Section 9.2]. This circle has no closed-form solution. EM breaks it by iteration: it fixes these point weights, the responsibilities of Section 3.1, solves for the parameters, and repeats.

3. The EM Algorithm

EM alternates two steps, each with a closed form for a Gaussian mixture. The E-step (expectation) computes, under the current parameters, how much each component is responsible for each point; the M-step (maximization) refits each component with these responsibilities as weights.

3.1. **The E-step.**

Given the current parameters, the E-step computes the *responsibility* of component k for the point x_i,

$$\gamma_{ik} = P(z_i = k \mid x_i) = \frac{\pi_k \, \mathcal{N}(x_i \mid \mu_k, \Sigma_k)}{\sum_{j=1}^{K} \pi_j \, \mathcal{N}(x_i \mid \mu_j, \Sigma_j)}. \tag{3.1}$$

In words, γ_{ik} is the share of component k in the density of x_i, so the responsibilities of each point are positive and add up to one.

Equation (3.1) is Bayes' rule for the hidden label z_i. The weight π_k is the prior, the probability of the label k before x_i is seen; the density $\mathcal{N}(x_i \mid \mu_k, \Sigma_k)$ is the likelihood, how well component k explains x_i; and the responsibility γ_{ik} is the posterior, the probability of the label k after x_i is seen. A point near the center of one component and far from the others gets a responsibility near one for that component; a point between two components is shared.

3.2. **The M-step.**

With the responsibilities fixed, the M-step computes new parameters for every component k:

$$\begin{aligned} N_k &= \sum_{i=1}^{n} \gamma_{ik}, \quad & \mu_k &= \frac{1}{N_k} \sum_{i=1}^{n} \gamma_{ik} \, x_i, \\ \pi_k &= \frac{N_k}{n}, & \Sigma_k &= \frac{1}{N_k} \sum_{i=1}^{n} \gamma_{ik} \, (x_i - \mu_k)(x_i - \mu_k)^{\mathsf{T}}. \end{aligned} \tag{3.2}$$

In words, every point counts toward component k with the weight γ_{ik}: N_k is the effective number of points of the component, π_k its share of the n points, μ_k the weighted mean of the points, and Σ_k their weighted covariance around the new mean.

These are the ordinary formulas for the proportion, the mean and the covariance of a group, with the hard label of each point, 0 or 1, replaced by the soft label γ_{ik} [3, Section 9.2]. They maximize the log-likelihood that the data would have if the labels were known, averaged over the labels with the responsibilities as their probabilities [3, 4]; this average is the expectation that names the E-step. The new weights add up to one, because the responsibilities of every point do.

3.3. **The algorithm.** Algorithm 1 repeats the two steps from a start $\theta^{(0)}$. The index t counts iterations, and $\theta^{(t)}$ denotes the parameters after t M-steps. EM stops when the log-likelihood changes by less than $\varepsilon = 10^{-10}$ from one iteration to the next, or after $T = 1000$ iterations.

Algorithm 1 EM for a Gaussian mixture

Input: points $x_1, \dots, x_n$; a start $\theta^{(0)}$; $\varepsilon = 10^{-10}$; at most $T = 1000$ iterations

1: compute $\ell(\theta^{(0)})$ by (2.2)
2: **for** $t = 0, 1, \dots, T-1$ **do**
3: E-step: γ_{ik} by (3.1) at $\theta^{(t)}$, for all i and k
4: M-step: $\theta^{(t+1)}$ by (3.2)
5: compute $\ell(\theta^{(t+1)})$ by (2.2)
6: **if** $|\ell(\theta^{(t+1)}) - \ell(\theta^{(t)})| < \varepsilon$ **then return** $\theta^{(t+1)}$ ▷ stopping test
7: **return** $\theta^{(T)}$

The notebook runs Algorithm 1 with the estimator `GaussianMixture` of scikit-learn [5], with full covariance matrices and three settings.

- `max_iter=1`: each call of the method `fit` performs one E-step and one M-step, lines 3 and 4.
- `warm_start=True`: each call continues from the parameters that the previous call left; the first starts from $\theta^{(0)}$, passed as the weights, the means and the inverse covariances.
- `reg_covar=0`: by default, scikit-learn adds a small number to the diagonal of every Σ_k, which keeps it invertible, and ℓ finite, when a component collapses onto too few points; without it, the M-step is (3.2), up to ten machine epsilons that scikit-learn adds to every N_k against a division by zero.

After each call, the notebook computes $\ell(\theta^{(t+1)})$ as n times the mean log-likelihood per point that the method `score` returns, and applies the stopping test of line 6 itself; $\ell(\theta^{(0)})$ comes from (2.2) directly, with SciPy.

3.4. **One iteration by hand.** In one dimension, $d = 1$, the covariance Σ is a variance σ^2, and $\mathcal{N}(x \mid \mu, \sigma^2) = e^{-(x-\mu)^2/(2\sigma^2)}/\sqrt{2\pi\sigma^2}$ is the bell curve. Take the four points $x = (1, 2, 6, 7)$, $K = 2$, and the start $\pi = (1/2, 1/2)$, $\mu = (0, 5)$ and $\sigma^2 = (1, 1)$. With equal weights and variances, (3.1) depends only on the ratio of the two densities, $\mathcal{N}(x \mid 5, 1)/\mathcal{N}(x \mid 0, 1) = e^{5x-12.5}$, so $\gamma_{i2} = 1/(1 + e^{12.5-5x_i})$ and $\gamma_{i1} = 1 - \gamma_{i2}$. Table 1 lists them: the points 1 and 2 belong mostly to the first component, and 6 and 7 to the second. Only $x = 2$, at distance 2 from μ_1 and 3 from μ_2, keeps a sizeable share of the second, 0.0759; $x = 1$ keeps 0.0006.

The M-step (3.2), computed from the unrounded responsibilities, gives the lower half of Table 1. The first component moves from 0 to 1.4804, between the points 1 and 2 and a little closer to 1, since 2 gave part of its weight away, and its variance shrinks to 0.2496. The second moves from 5 to 6.3341: the small share of the far point 2 holds it back from the middle of 6 and 7 and keeps its variance near one, at 0.9611. The log-likelihood (2.2) rises from -11.3689 to -6.3155, consistent with Theorem 4.1 of Section 4.

TABLE 1. One iteration of EM by hand, for the points $1, 2, 6, 7$ and the start $\pi = (1/2, 1/2)$, $\mu = (0, 5)$, $\sigma^2 = (1, 1)$, to four decimals.

	$k = 1$	$k = 2$
E-step: responsibilities (3.1)		
γ_{1k}, $x_1 = 1$	0.9994	0.0006
γ_{2k}, $x_2 = 2$	0.9241	0.0759
γ_{3k}, $x_3 = 6$	0.0000	1.0000
γ_{4k}, $x_4 = 7$	0.0000	1.0000
M-step: new parameters (3.2)		
N_k	1.9236	2.0764
π_k	0.4809	0.5191
μ_k	1.4804	6.3341
σ_k^2	0.2496	0.9611

4. Why EM Works

Theorem 4.1 (Monotone ascent [2])**.** *Every iteration of Algorithm 1 satisfies* $\ell(\theta^{(t+1)}) \geq \ell(\theta^{(t)})$.

In words, EM never lowers the log-likelihood: every iteration climbs, or stays at the same height.

Proof idea. Keep the responsibilities γ_{ik} of the E-step at $\theta^{(t)}$ fixed, and let $B(\theta)$ be $\ell(\theta)$ with the logarithm of each inner sum of (2.2), an average of the terms $\pi_k \mathcal{N}(x_i \mid \mu_k, \Sigma_k)/\gamma_{ik}$ with the weights γ_{ik}, replaced by the average of their logarithms.

(i) Jensen's inequality gives $B(\theta) \leq \ell(\theta)$ for every θ: the logarithm is concave, so the logarithm of an average is at least the average of the logarithms.

(ii) B touches ℓ at $\theta^{(t)}$, by (3.1): there, all the averaged terms of a point are equal, so $B(\theta^{(t)}) = \ell(\theta^{(t)})$.

(iii) The M-step maximizes B, which is, up to a constant, the averaged log-likelihood with known labels of Section 3.2.

Hence $\ell(\theta^{(t+1)}) \geq B(\theta^{(t+1)}) \geq B(\theta^{(t)}) = \ell(\theta^{(t)})$. □

Bishop [3] draws this argument as a picture: a curve below ℓ that touches it at $\theta^{(t)}$ and whose highest point lies at $\theta^{(t+1)}$. The E-step builds the curve, and the M-step climbs to its top.

Theorem 4.1 says that EM climbs, not where it arrives. Under regularity conditions, Wu [6] proved that the values $\ell(\theta^{(t)})$ converge and that every limit point of the iterates is a *stationary point* of ℓ, a point where its gradient vanishes. EM thus settles where the log-likelihood is flat: at a local maximum, or at a *saddle point*, a flat point from which ℓ rises in some directions and falls in others.

Which stationary point EM reaches depends on the start, and a mixture has many. Swapping the labels of two components gives the same ℓ, so every maximum comes in $K!$ copies; different groupings of the data give different local maxima; and two identical components stay identical forever, since every responsibility then equals the weight π_k and the M-step gives both the mean

and the covariance of all the points. EM is therefore run from several starts, and the fit with the largest ℓ among those with no collapsed component is kept [4].

The stopping test of Algorithm 1 checks only that ℓ rose by less than ε in the last iteration, which does not prove a maximum. Near a saddle point, such as two identical components, ℓ can rise slowly for many iterations before it climbs fast (Section 5.2). The tolerance must be small compared with such slow rises, and the cap of T iterations ends every run.

5. Experiments

The estimator `GaussianMixture` of scikit-learn is the method, and the hand iteration of Section 3.4 is the check. The runs of Sections 5.2 and 5.3 use Algorithm 1 as Section 3.3 describes it, with full covariances, `reg_covar=0`, $\varepsilon = 10^{-10}$ and at most 1000 iterations. Only the random starts of Section 5.3 use random numbers, from a fixed seed, so every number below is deterministic.

5.1. **The hand iteration against scikit-learn.** The scikit-learn estimator `GaussianMixture` reproduces the hand iteration to machine precision. One call of `fit`, with `max_iter=1` and `reg_covar=0`, from the start of Section 3.4, gives the weights, means and variances of Table 1; the largest difference from the hand values, computed with NumPy and SciPy from (3.1) and (3.2), is $6.2 \cdot 10^{-15}$. The log-likelihood that scikit-learn reports after the step is -6.3155, the hand value. One call of `fit` is therefore one iteration of Algorithm 1; with `warm_start=True`, every later call runs the same two steps from the current parameters.

5.2. **Old Faithful.** From a poor start, EM first makes the two components almost identical, leaves this state slowly, and then separates the short eruptions from the long ones, with ℓ rising at every iteration. The 272 eruptions are standardized: each variable is centered at its mean and divided by its standard deviation, the units of Figure 1. Every value of ℓ here and in Section 5.3 is that of the standardized data, and Table 2 converts the fit back to minutes. The start has equal weights, identity covariances, and the means $(-1.5, 1.5)$ and $(1.5, -1.5)$, where there are no points (Figure 1(a)).

At the start, most points have a sizeable responsibility for both components, so the first M-step gives both nearly the mean and the covariance of all the

Table 2. The mixture fitted to the 272 Old Faithful eruptions, in the original units: the weights π_k, the means of eruption time and waiting time in minutes, and the covariance entries in minutes squared, the variances σ_{e}^2 of eruption time and σ_{w}^2 of waiting time, and their covariance σ_{ew}.

		mean		covariance		
k	π_k	eruption	waiting	σ_{e}^2	σ_{ew}	σ_{w}^2
1	0.356	2.04	54.48	0.069	0.435	33.697
2	0.644	4.29	79.97	0.170	0.941	36.046

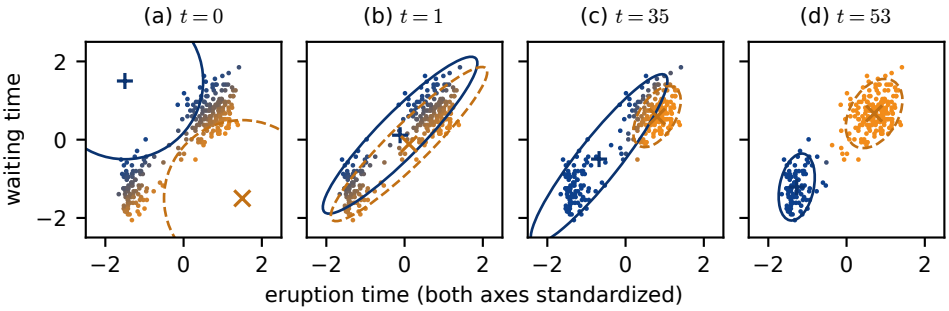


FIGURE 1. EM on the standardized Old Faithful data, from equal weights, identity covariances and the means $(-1.5, 1.5)$ and $(1.5, -1.5)$, in the style of [3, Fig. 9.8]: (a) the start, (b) after one iteration, (c) after 35, when ℓ has covered half of its climb, (d) converged; + and × mark the means. Each point is colored by its responsibilities, dark blue for the first component, orange for the second, and a mix when shared. The ellipses lie two standard deviations from the means: solid for the first component, dashed for the second.

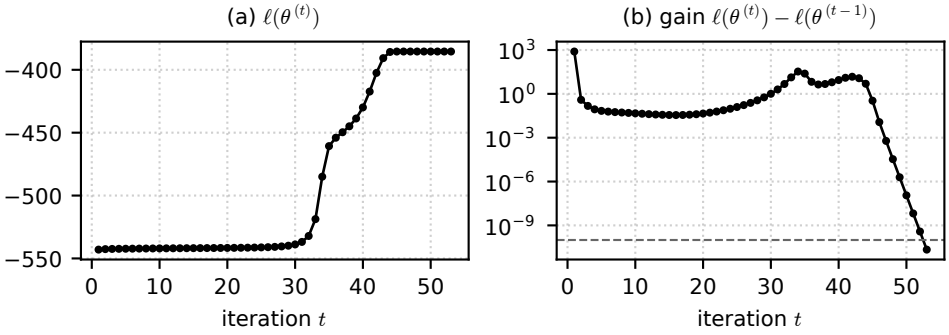


FIGURE 2. The Old Faithful run of Figure 1: (a) the log-likelihood $\ell(\theta^{(t)})$ for $t \geq 1$; (b) the gain $\ell(\theta^{(t)}) - \ell(\theta^{(t-1)})$ of each iteration, on a log scale, with the tolerance $\varepsilon = 10^{-10}$ dashed. Panel (a) leaves out the start, $\ell(\theta^{(0)}) = -1331.48$, for scale. Every gain is positive, small while the two components almost coincide and large once they separate; the last falls below ε and ends the run.

points. The two ellipses almost coincide (Figure 1(b)), and ℓ jumps from -1331.48 to -542.98.

This is close to the identical components of Section 4, and EM leaves them slowly (Figure 2(a)): ℓ covers half of its climb from -542.98 to -385.4607 only at iteration 35 (Figure 1(c)). On the way, ℓ still rose by at least 0.035 per iteration, far more than ε, so the stopping test did not end the run on this plateau. Once the components separate, the climb is fast, and EM stops after 53 iterations, at $\ell = -385.4607$ (Figure 1(d)). The smallest gain of ℓ in one iteration is $2.2 \cdot 10^{-11}$, positive (Figure 2(b)), so ℓ rose at every iteration, consistent with Theorem 4.1, which requires only that it never falls.

The two components are the two groups of eruptions. The first, with weight 0.356, holds the short eruptions followed by short waits, the second the long ones (Table 2).

5.3. **Twenty random starts.** Twenty random starts all reach the log-likelihood of the poor start; on these data, the start changes the number of iterations, not the final ℓ. Each start has equal weights, identity covariances, and the means at two data points chosen at random, with a fixed seed. Every run stops at $\ell = -385.4607$, to four decimals, after 11 to 37 iterations, fewer than the 53 of the poor start. None of them ends at a stationary point with a different log-likelihood, although Section 4 allows it. None of the 21 final fits, the poor start's and these twenty, collapses: the smallest covariance eigenvalue, the smallest variance of a component in any direction, is 0.047, in standardized units.

6. Conclusion

(1) EM fits a Gaussian mixture by alternating Bayes' rule for the hidden labels, the E-step (3.1), with weighted versions of the ordinary Gaussian fits, the M-step (3.2).
(2) One call of scikit-learn's `fit` reproduces the hand iteration to $6.2 \cdot 10^{-15}$, so the experiments run EM as derived.
(3) On Old Faithful, ℓ rose at every iteration, consistent with Theorem 4.1; from a poor start, EM lingered near two identical components and converged after 53 iterations.
(4) All twenty random starts reached the same log-likelihood, -385.4607, in 11 to 37 iterations.

The findings have limits: one data set, two components, one poor start and twenty random ones; other data, more components or other starts can end at other stationary points.

The same alternation of two steps underlies hidden Markov models and linear-Gaussian state-space models, where the E-step is a forward-backward recursion or a Kalman smoother [3, Chapter 13]. Todorović, Todorović-Zarkula and Stanković [7] treat Kalman-filter estimation of the states and parameters of recurrent neural networks.

Code availability

The notebook `em_project.ipynb`, in the folder `project`, computes every number, table and figure in Sections 3.4 and 5: NumPy and SciPy compute the hand iteration, and scikit-learn runs EM. It runs on Python 3.14.7 with NumPy 2.5.3, SciPy 1.18.1, scikit-learn 1.9.1 and Matplotlib 3.11.2; these four, with ipykernel and nbconvert for the run command, are pinned in `project/requirements.txt`. With them installed in a virtual environment `project/.venv`, the commands

```
cd project
.venv/bin/jupyter nbconvert --to notebook --execute --inplace em_project.ipynb
```

run it; the run reported here took less than a minute. The public repository https://github.com/innicoder/em-algorithm-seminar holds this paper, its LaTeX source, the notebook with its outputs, and the figures. The data

file `faithful.csv` in `project/data` is the data set `faithful` of R, from the Rdatasets collection.

References

[1] A. Azzalini and A. W. Bowman. "A look at some data on the Old Faithful geyser". In: *Applied Statistics* 39.3 (1990), pp. 357–365. DOI: 10.2307/2347385.

[2] A. P. Dempster, N. M. Laird, and D. B. Rubin. "Maximum likelihood from incomplete data via the EM algorithm". In: *Journal of the Royal Statistical Society: Series B (Methodological)* 39.1 (1977), pp. 1–22. DOI: 10.1111/j.2517-6161.1977.tb01600.x.

[3] C. M. Bishop. *Pattern Recognition and Machine Learning*. Information Science and Statistics. New York: Springer, 2006.

[4] G. J. McLachlan and T. Krishnan. *The EM Algorithm and Extensions*. 2nd ed. Wiley Series in Probability and Statistics. Hoboken, NJ: John Wiley & Sons, 2008. DOI: 10.1002/9780470191613.

[5] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay. "Scikit-learn: Machine learning in Python". In: *Journal of Machine Learning Research* 12.85 (2011), pp. 2825–2830.

[6] C. F. J. Wu. "On the convergence properties of the EM algorithm". In: *The Annals of Statistics* 11.1 (1983), pp. 95–103. DOI: 10.1214/aos/1176346060.

[7] B. Todorović, S. Todorović-Zarkula, and M. Stanković. *Rekurentne neuronske mreže: estimacija parametara, stanja i strukture*. Niš: Univerzitet u Nišu, Prirodno-matematički fakultet, 2012.